

AEROSENTINEL

Flight Data Intelligence Platform

A Complete 3-Week Software Engineering Portfolio Project

3 Weeks Total Timeline	~28 Hours 8-10 hrs / week	4 Domains C++, Python, ML, DevOps
----------------------------------	-------------------------------------	--

Domains Covered:

Systems Programming • Data Engineering • Machine Learning • Cybersecurity • DevOps • Avionics/Aerospace

Targeted for: Boeing Defense, Space & Security — Software Engineering Roles

Section 1: Project Overview

1.1 What is AeroSentinel?

AeroSentinel is a real-time avionics telemetry pipeline with ML-powered anomaly detection and a live monitoring dashboard. It simulates the kind of embedded data infrastructure used in defense aircraft, UAVs, and space systems — and it touches every major skill area a Boeing defense software engineer needs to demonstrate.

The system has three layers:

- A C++ engine that receives, validates, and buffers binary telemetry packets from a simulated aircraft sensor suite
- A Python analytics layer that processes the data, runs an Isolation Forest anomaly detection model, persists everything to a time-series database, and exposes it via a REST API
- A Grafana live dashboard and Docker Compose deployment that makes the whole system reproducible and demo-ready

1.2 Why this project works for Boeing

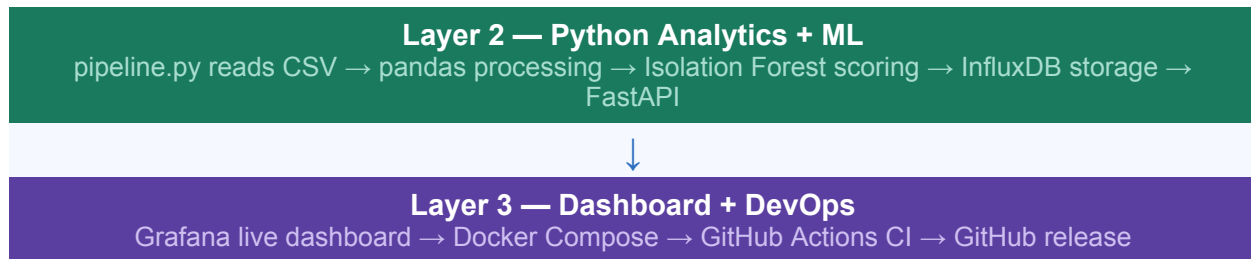
Boeing Defense & Space interviews consistently test for four things: systems-level C++ thinking, data pipeline design, applied ML, and deployment maturity. This project demonstrates all four with real artifacts — not toy exercises.

Skill Area	What You Build	Boeing Relevance
C++ Systems	UDP receiver, binary parser, ring buffer	Embedded SW, avionics firmware
Data Engineering	Pandas pipeline, InfluxDB, FastAPI	Telemetry processing, SCADA
Machine Learning	Isolation Forest anomaly detection	Predictive maintenance, health monitoring
DevOps / CI	Docker Compose, GitHub Actions	Reproducible builds, DO-178C readiness
Security	API auth, input validation, SECURITY.md	Classified systems awareness

1.3 System Architecture

The data flows in one direction through each layer, with no circular dependencies. This is intentional — it mirrors how real avionics data buses are designed.





1.4 How to use this guide

Each section maps to one week of work. Within each week, tasks are organized by day range and estimate. Work through them in order — each one builds directly on the previous.

- Read the task description fully before writing any code
- The 'Learning moment' boxes explain the why behind each task — read them, they prepare you for interviews
- Each 'Deliverable' box tells you exactly what 'done' looks like before moving on
- Code blocks show exact commands to run in your terminal
- Commit to GitHub at the end of every session — this is part of the project

Section 2: Full Environment Setup

2.1 What to install (do this first, before anything else)

Set aside 1-2 hours the first day purely for setup. A solid environment prevents frustrating bugs later. Install everything in the order listed below.

2.1.1 Core Development Tools

Tool	How to Install / Where to Get It
VSCode	https://code.visualstudio.com — free, download the installer for your OS
Git	https://git-scm.com — or via Homebrew on Mac: <code>brew install git</code>
CMake 3.20+	https://cmake.org/download — or: <code>brew install cmake</code> / <code>sudo apt install cmake</code>
GCC or Clang	Mac: <code>xcode-select --install</code> Linux: <code>sudo apt install build-essential</code>
Python 3.11+	https://python.org or via <code>pyenv</code> . Verify: <code>python3 --version</code>
Docker Desktop	https://docker.com/products/docker-desktop — free for personal use
Node.js (optional)	Only needed if you add JS tooling. https://nodejs.org

2.1.2 VSCode Extensions

Open VSCode, press `Ctrl+Shift+X` (or `Cmd+Shift+X` on Mac) and install each of these:

Extension Name	What It Does
C/C++ (Microsoft)	Syntax highlighting, IntelliSense, and debugging for C/C++
CMake Tools (Microsoft)	Integrates CMake build system directly into VSCode sidebar
Python (Microsoft)	Python language support, linting, and virtual environment management
Docker (Microsoft)	View containers, images, and compose files inside VSCode
GitLens	Inline git blame, history, and repository visualization
REST Client	Test your FastAPI endpoints directly from <code>.http</code> files inside VSCode
YAML	Syntax support for <code>docker-compose.yml</code> and GitHub Actions workflows

2.1.3 Python Packages

Create a virtual environment for the project and install all dependencies at once:

```
# Navigate to your project folder first
```

```
python3 -m venv venv
source venv/bin/activate      # Mac/Linux
venv\Scripts\activate        # Windows

# Install all Python dependencies
pip install pandas numpy scikit-learn joblib influxdb-client fastapi uvicorn

# Save exact versions for reproducibility
pip freeze > requirements.txt
```

2.1.4 C++ Dependencies

You need Google Test (gtest) for unit testing. The easiest approach is to use CMake's FetchContent to download it automatically at build time — you will set this up in Week 1.

Learning moment

A virtual environment isolates your Python packages so they don't conflict with system Python. Pinning versions in requirements.txt is a security and reproducibility practice — it means anyone who clones your repo gets the exact same library versions you tested with. This is how production software is managed.

2.2 Create your GitHub repository

Do this before writing a single line of project code. Your commit history is part of the portfolio.

1. Go to github.com and create a new public repository named aerosentinel
2. Clone it locally: `git clone https://github.com/YOUR_USERNAME/aerosentinel.git`
3. Create the folder structure shown below, then commit and push it immediately

```
aerosentinel/
  src/           # C++ source files
  include/       # C++ header files
  tests/         # C++ unit tests (gtest)
  scripts/       # Python scripts (simulator, pipeline, api)
  data/          # Generated CSV files (add to .gitignore)
  docker/        # Docker Compose and Grafana provisioning
    grafana/
      provisioning/
        datasources/
        dashboards/
  models/        # Saved ML model files
  .github/
    workflows/   # GitHub Actions CI
```

```
README.md  
requirements.txt  
CMakeLists.txt  
.gitignore
```

Add data/ to .gitignore

The telemetry CSV files will grow large and should not be committed. Add `data/*.csv` and `data/*.log` to your `.gitignore` file right now.

Section 3: Week 1 — C++ Telemetry Pipeline

This week you build the core of the system: a Python-based aircraft sensor simulator and a C++ ground station that receives, validates, stores, and writes the data. This is the most technically impressive part of the project from a Boeing hiring perspective.

Week 1 Goal

By the end of this week: C++ program receives binary UDP packets at 10Hz, validates each one with a CRC checksum, stores them in a ring buffer, and writes clean CSV output. All unit tests pass with zero memory errors.

Days 1–2

CMake project scaffold + Hello World build

~2.5 hrs

Step 1: Create CMakeLists.txt

This is the build configuration file that tells CMake how to compile your C++ code. Create it at the root of your project:

```
cmake_minimum_required(VERSION 3.20)
project(AeroSentinel VERSION 1.0)

set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)

# Enable AddressSanitizer in Debug builds for memory checking
if(CMAKE_BUILD_TYPE STREQUAL "Debug")
    add_compile_options(-fsanitize=address -fno-omit-frame-pointer)
    add_link_options(-fsanitize=address)
endif()

# Fetch Google Test automatically
include(FetchContent)
FetchContent_Declare(
    googletest
    URL https://github.com/google/googletest/archive/v1.14.0.zip
)
FetchContent_MakeAvailable(googletest)

# Main executable
add_executable(aerosentinel src/main.cpp src/parser.cpp src/ring_buffer.cpp)
target_include_directories(aerosentinel PRIVATE include)

# Test executable
```

```
enable_testing()
add_executable(aero_tests tests/test_parser.cpp tests/test_ring_buffer.cpp
               src/parser.cpp src/ring_buffer.cpp)
target_include_directories(aero_tests PRIVATE include)
target_link_libraries(aero_tests GTest::gtest_main)
include(GoogleTest)
gtest_discover_tests(aero_tests)
```

Step 2: Write a Hello World main.cpp to verify the build

Create src/main.cpp with just enough code to confirm CMake compiles successfully:

```
#include <iostream>

int main() {
    std::cout << "AeroSentinel: systems online" << std::endl;
    return 0;
}
```

Step 3: Build and run

```
# From your project root:
mkdir build && cd build
cmake .. -DCMAKE_BUILD_TYPE=Debug
make -j4

# Run it:
./aerosentinel

# Expected output: AeroSentinel: systems online
```

Learning moment

CMake is the industry-standard build system for C++ in aerospace, automotive, and defense. Every major codebase Boeing, Raytheon, and Lockheed works with uses it. Understanding CMake targets, build types, and FetchContent puts you ahead of most junior candidates.

Deliverable: CMake project compiles cleanly and prints output. Committed to GitHub.

Days 2–4

Telemetry struct + Python simulator + C++ UDP receiver

~4 hrs

Step 4: Define the telemetry packet structure

Create `include/telemetry.h` — this defines the binary layout of every packet sent between the simulator and receiver. Both sides must use the exact same struct:

```
#pragma once
#include <cstdint>
#include <ctime>

// Packed struct ensures no compiler padding between fields.
// This is critical for binary protocols where byte layout must be exact.
#pragma pack(push, 1)
struct TelemetryFrame {
    uint32_t sequence_number; // Packet counter (detect dropped packets)
    uint64_t timestamp_us;    // Microseconds since epoch
    float altitude_ft;        // Altitude in feet
    float airspeed_kts;       // Indicated airspeed in knots
    float heading_deg;        // Magnetic heading 0-360
    float engine_rpm;         // Engine RPM
    float fuel_pct;           // Fuel level 0-100%
    float latitude;           // GPS latitude
    float longitude;          // GPS longitude
    uint8_t fault_flags;      // Bit flags for injected faults
    uint16_t crc;             // CRC-16 checksum of all preceding bytes
};
#pragma pack(pop)

// Total size should be exactly 43 bytes.
// Verify at compile time:
static_assert(sizeof(TelemetryFrame) == 43, "TelemetryFrame size mismatch");
```

Learning moment

The `#pragma pack(1)` directive forces the compiler to lay out struct fields with no padding bytes between them. Without this, a struct with a float followed by a double might have 4 hidden bytes inserted for alignment. In binary protocols, both sides must agree on the exact byte layout. ARINC 429 and MIL-STD-1553 avionics buses use this principle. The `static_assert` catches size mismatches at compile time rather than at runtime — a safety technique used in safety-critical software.

Step 5: Implement CRC-16 checksum

Create `src/parser.cpp` and `include/parser.h`. The CRC detects packet corruption — a receiver that accepts corrupted data silently is a safety hazard:

```
// include/parser.h
#pragma once
#include "telemetry.h"
```

```
#include <vector>
#include <stdint>

uint16_t crc16(const uint8_t* data, size_t length);
bool      validate_frame(const TelemetryFrame& frame);
bool      parse_bytes(const uint8_t* buf, size_t len, TelemetryFrame& out);
```

```
// src/parser.cpp
#include "parser.h"
#include <cstring>

// CRC-16/CCITT-FALSE - widely used in aerospace and industrial protocols
uint16_t crc16(const uint8_t* data, size_t length) {
    uint16_t crc = 0xFFFF;
    for (size_t i = 0; i < length; ++i) {
        crc ^= static_cast<uint16_t>(data[i]) << 8;
        for (int j = 0; j < 8; ++j) {
            if (crc & 0x8000) crc = (crc << 1) ^ 0x1021;
            else               crc <<= 1;
        }
    }
    return crc;
}

bool validate_frame(const TelemetryFrame& frame) {
    // Compute CRC over all bytes EXCEPT the last 2 (the crc field itself)
    size_t data_len = sizeof(TelemetryFrame) - sizeof(uint16_t);
    uint16_t computed = crc16(reinterpret_cast<const uint8_t*>(&frame), data_len);
    return computed == frame.crc;
}

bool parse_bytes(const uint8_t* buf, size_t len, TelemetryFrame& out) {
    if (len < sizeof(TelemetryFrame)) return false;
    std::memcpy(&out, buf, sizeof(TelemetryFrame));
    return validate_frame(out);
}
```

Step 6: Write the Python telemetry simulator

Create scripts/simulator.py. This plays the role of an aircraft's avionics computer sending sensor data over UDP:

```
import socket, struct, time, math, random, argparse

# Struct format must match TelemetryFrame exactly: I Q 7f B H
```

```
# I=uint32, Q=uint64, f=float (x7), B=uint8, H=uint16
FMT = '<IQ7fBH' # little-endian

def crc16(data: bytes) -> int:
    crc = 0xFFFF
    for byte in data:
        crc ^= byte << 8
        for _ in range(8):
            crc = (crc << 1) ^ 0x1021 if crc & 0x8000 else crc << 1
    return crc & 0xFFFF

def make_packet(seq: int, inject_fault: str = None) -> bytes:
    t = time.time()
    alt = 5000 + 200 * math.sin(t / 30) # Gentle altitude oscillation
    speed = 250 + 10 * math.sin(t / 20)
    hdg = (t * 2) % 360
    rpm = 2400 + 50 * math.sin(t / 10)
    fuel = max(0, 80 - (t % 3600) / 3600 * 80)
    lat = 47.6062 + 0.001 * math.sin(t / 60)
    lon = -122.3321 + 0.001 * math.cos(t / 60)
    flags = 0

    # Fault injection for testing anomaly detection
    if inject_fault == 'engine_spike':
        rpm = 4800; flags |= 0x01
    elif inject_fault == 'altitude_drop':
        alt -= 2000; flags |= 0x02
    elif inject_fault == 'gps_freeze':
        lat = 47.6062; lon = -122.3321; flags |= 0x04
    elif inject_fault == 'sensor_dropout':
        alt = speed = rpm = 0.0; flags |= 0x08

    ts_us = int(t * 1_000_000)
    # Pack all fields except CRC
    body = struct.pack('<IQ7fBH', seq, ts_us, alt, speed, hdg, rpm, fuel, lat, lon,
flags)
    crc = crc16(body)
    return body + struct.pack('<H', crc)

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument('--fault', choices=['engine_spike', 'altitude_drop',
'gps_freeze', 'sensor_dropout'], default=None)
    parser.add_argument('--corrupt', action='store_true',
help='Randomly corrupt 2%% of packets to test CRC')
```

```
args = parser.parse_args()

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
seq = 0
print(f'Simulator running. Fault={args.fault}, Corrupt={args.corrupt}')
while True:
    pkt = make_packet(seq, args.fault)
    if args.corrupt and random.random() < 0.02:
        pkt = bytes([b ^ 0xFF if i == 5 else b for i, b in enumerate(pkt)])
    sock.sendto(pkt, ('127.0.0.1', 5005))
    seq += 1
    time.sleep(0.1) # 10 Hz

if __name__ == '__main__':
    main()
```

Step 7: Write the C++ UDP receiver in main.cpp

Replace the Hello World main.cpp with the real receiver:

```
#include <iostream>
#include <fstream>
#include <sstream>
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>
#include <chrono>
#include "telemetry.h"
#include "parser.h"
#include "ring_buffer.h"

int main() {
    // Create UDP socket
    int sock = socket(AF_INET, SOCK_DGRAM, 0);
    if (sock < 0) { perror("socket"); return 1; }

    sockaddr_in addr{};
    addr.sin_family = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;
    addr.sin_port = htons(5005);

    if (bind(sock, (sockaddr*)&addr, sizeof(addr)) < 0) {
        perror("bind"); return 1;
    }
}
```

```
RingBuffer<TelemetryFrame, 512> buffer;
std::ofstream csv("data/telemetry.csv");
csv << "seq,timestamp_us,altitude_ft,airspeed_kts,heading_deg,"
    << "engine_rpm,fuel_pct,latitude,longitude,fault_flags\n";

uint8_t raw[256];
uint32_t ok = 0, bad = 0;
auto last_flush = std::chrono::steady_clock::now();

std::cout << "AeroSentinel receiver listening on :5005\n";

while (true) {
    ssize_t n = recv(sock, raw, sizeof(raw), 0);
    if (n < 0) continue;

    TelemetryFrame frame;
    if (parse_bytes(raw, n, frame)) {
        buffer.push(frame);
        ++ok;
        std::cout << "[OK  #" << frame.sequence_number
            << "] alt=" << frame.altitude_ft
            << " spd=" << frame.airspeed_kts
            << " rpm=" << frame.engine_rpm << "\n";
    } else {
        ++bad;
        std::cerr << "[BAD] CRC mismatch - packet rejected (" +
            std::to_string(bad) + " total)\n";
    }

    // Flush buffer to CSV every 5 seconds
    auto now = std::chrono::steady_clock::now();
    if
    (std::chrono::duration_cast<std::chrono::seconds>(now-last_flush).count() >= 5) {
        while (!buffer.empty()) {
            auto f = buffer.pop();
            csv << f.sequence_number << "," << f.timestamp_us << ","
                << f.altitude_ft << "," << f.airspeed_kts << ","
                << f.heading_deg << "," << f.engine_rpm << ","
                << f.fuel_pct << "," << f.latitude << ","
                << f.longitude << "," << (int)f.fault_flags << "\n";
        }
        csv.flush();
        last_flush = now;
    }
}
```

```
    close(sock);  
}
```

Step 8: Implement the ring buffer

Create `include/ring_buffer.h`. This is a fixed-size circular buffer — it overwrites the oldest entry when full, which is the correct behavior for a real-time sensor system that must never block:

```
#pragma once  
#include <array>  
#include <stdexcept>  
  
template<typename T, size_t N>  
class RingBuffer {  
    std::array<T, N> buf_;  
    size_t head_ = 0;  
    size_t tail_ = 0;  
    size_t size_ = 0;  
public:  
    void push(const T& item) {  
        buf_[head_] = item;  
        head_ = (head_ + 1) % N;  
        if (size_ < N) ++size_;  
        else tail_ = (tail_ + 1) % N; // Overwrite oldest  
    }  
    T pop() {  
        if (size_ == 0) throw std::underflow_error("Buffer empty");  
        T item = buf_[tail_];  
        tail_ = (tail_ + 1) % N;  
        --size_;  
        return item;  
    }  
    bool empty() const { return size_ == 0; }  
    size_t size() const { return size_; }  
    size_t capacity() const { return N; }  
};
```

Learning moment

A ring buffer (circular buffer) is one of the most important data structures in embedded and real-time systems. It provides $O(1)$ push and pop with no dynamic memory allocation. In avionics, flight data recorders (FDRs) use this pattern: when the buffer fills, the oldest data is overwritten with new data, ensuring the most recent flight data is always preserved. FIFO queues backed by ring buffers appear in RTOS kernels, network drivers, and audio processing pipelines.

Days 5–7

Google Test unit tests + AddressSanitizer

~3.5 hrs

Step 9: Write unit tests with Google Test

Create tests/test_parser.cpp. Writing tests before you're sure the code is correct is how professionals find bugs:

```
#include <gtest/gtest.h>
#include "parser.h"
#include "telemetry.h"
#include <cstring>

// Helper: build a valid packet byte array
std::vector<uint8_t> make_valid_packet(uint32_t seq = 1) {
    TelemetryFrame f{};
    f.sequence_number = seq;
    f.timestamp_us     = 1700000000000000ULL;
    f.altitude_ft      = 5000.0f;
    f.airspeed_kts     = 250.0f;
    f.heading_deg      = 090.0f;
    f.engine_rpm       = 2400.0f;
    f.fuel_pct         = 75.0f;
    f.latitude         = 47.6f;
    f.longitude        = -122.3f;
    f.fault_flags      = 0;
    // Compute and set correct CRC
    size_t data_len = sizeof(TelemetryFrame) - sizeof(uint16_t);
    f.crc = crc16(reinterpret_cast<const uint8_t*>(&f), data_len);
    std::vector<uint8_t> buf(sizeof(TelemetryFrame));
    std::memcpy(buf.data(), &f, sizeof(TelemetryFrame));
    return buf;
}

TEST(ParserTest, ValidPacketParsesCorrectly) {
    auto buf = make_valid_packet(42);
    TelemetryFrame out;
    EXPECT_TRUE(parse_bytes(buf.data(), buf.size(), out));
    EXPECT_EQ(out.sequence_number, 42u);
    EXPECT_FLOAT_EQ(out.altitude_ft, 5000.0f);
}

TEST(ParserTest, CorruptedPacketFailsCRC) {
    auto buf = make_valid_packet();
    buf[5] ^= 0xFF; // Corrupt one byte
```

```
    TelemetryFrame out;
    EXPECT_FALSE(parse_bytes(buf.data(), buf.size(), out));
}

TEST(ParserTest, TooShortBufferReturnsFalse) {
    std::vector<uint8_t> buf(10, 0x00);
    TelemetryFrame out;
    EXPECT_FALSE(parse_bytes(buf.data(), buf.size(), out));
}
```

Step 10: Write ring buffer tests

Create tests/test_ring_buffer.cpp:

```
#include <gtest/gtest.h>
#include "ring_buffer.h"
#include "telemetry.h"

TEST(RingBufferTest, PushAndPopSingleItem) {
    RingBuffer<int, 4> buf;
    buf.push(99);
    EXPECT_EQ(buf.size(), 1u);
    EXPECT_EQ(buf.pop(), 99);
    EXPECT_TRUE(buf.empty());
}

TEST(RingBufferTest, OverflowOverwritesOldest) {
    RingBuffer<int, 3> buf;
    buf.push(1); buf.push(2); buf.push(3);
    buf.push(4); // Should overwrite 1
    EXPECT_EQ(buf.size(), 3u);
    EXPECT_EQ(buf.pop(), 2); // 1 was overwritten
}

TEST(RingBufferTest, PopOnEmptyThrows) {
    RingBuffer<int, 4> buf;
    EXPECT_THROW(buf.pop(), std::underflow_error);
}
```

Step 11: Build and run tests with AddressSanitizer

```
cd build
cmake .. -DCMAKE_BUILD_TYPE=Debug
make -j4
```



```
# Run the tests
ctest --verbose

# Or run the test binary directly to see all output
./aero_tests

# Expected: All tests pass, no memory errors reported by ASan
```

Learning moment

AddressSanitizer (ASan) is a compile-time instrumentation tool that catches memory bugs at runtime: buffer overflows, use-after-free, and stack corruptions. It adds roughly 2x runtime overhead but catches entire classes of bugs that would otherwise only appear in production. Mentioning 'I run my tests under AddressSanitizer as part of the debug build' in a Boeing interview signals professional-level safety awareness.

Deliverable: All gtest tests pass. C++ receiver prints validated packets to console at 10Hz. CSV file written every 5 seconds. Zero AddressSanitizer errors. Committed to GitHub.

Section 4: Week 2 — Python Analytics & ML Layer

This week you build the data science layer. You'll stand up InfluxDB, process telemetry with pandas, train an Isolation Forest anomaly detection model, and expose everything via a FastAPI REST API. By the end you'll have a fully functional anomaly detection engine running against live data.

Week 2 Goal

By the end of this week: Python pipeline reads CSV, processes data, detects anomalies with ML, writes results to InfluxDB, and serves them via FastAPI. All four injected fault types are detected and logged.

Days 1–3

InfluxDB setup + Pandas pipeline

~4 hrs

Step 12: Start InfluxDB with Docker

Run this command once to start InfluxDB. Docker pulls the image automatically on first run:

```
docker run -d \
  --name influxdb \
  -p 8086:8086 \
  -v influxdb-data:/var/lib/influxdb2 \
  influxdb:2.7

# Open http://localhost:8086 in your browser
# Complete the setup wizard:
#   Username: admin
#   Password: aerosentinel123
#   Organization: aerosentinel-org
#   Bucket: aerosentinel
#
# Generate an API token from the UI: Data > API Tokens > Generate
# Save it - you will need it in the next step
```

Step 13: Write the Python data pipeline

Create scripts/pipeline.py. This is the bridge between the C++ output and the analytics/storage layer:

```
import time, os, joblib
import pandas as pd
import numpy as np
from influxdb_client import InfluxDBClient, Point
from influxdb_client.client.write_api import SYNCHRONOUS
```

```
INFLUX_URL    = 'http://localhost:8086'
INFLUX_TOKEN  = os.environ.get('INFLUX_TOKEN', 'YOUR_TOKEN_HERE')
INFLUX_ORG    = 'aerosentinel-org'
INFLUX_BUCKET = 'aerosentinel'
CSV_PATH      = 'data/telemetry.csv'

client        = InfluxDBClient(url=INFLUX_URL, token=INFLUX_TOKEN, org=INFLUX_ORG)
write_api     = client.write_api(write_options=SYNCHRONOUS)

# Load the saved anomaly detection model (trained in Step 16)
model = None
if os.path.exists('models/anomaly_model.joblib'):
    model = joblib.load('models/anomaly_model.joblib')
    print('Anomaly detection model loaded.')
else:
    print('No model found. Run train_model.py first, then restart.')

FEATURES = ['altitude_ft', 'airspeed_kts', 'engine_rpm',
            'rate_of_climb', 'heading_change_rate']

last_row = 0 # Track how far we have read in the CSV

def compute_derived(df: pd.DataFrame) -> pd.DataFrame:
    df = df.sort_values('timestamp_us').copy()
    df['dt'] = df['timestamp_us'].diff() / 1e6 # seconds
    df['rate_of_climb'] = df['altitude_ft'].diff() / df['dt'].replace(0, np.nan)
    df['heading_change_rate'] = df['heading_deg'].diff().abs() /
    df['dt'].replace(0, np.nan)
    return df.fillna(0)

def process_batch(df: pd.DataFrame):
    global model
    df = compute_derived(df)
    anomaly_scores = np.zeros(len(df))
    is_anomaly = np.zeros(len(df), dtype=bool)
    if model is not None:
        X = df[FEATURES].values
        anomaly_scores = model.decision_function(X)
        is_anomaly = model.predict(X) == -1 # -1 = anomaly in sklearn
    for _, row in df.iterrows():
        idx = df.index.get_loc(row.name)
        point = (Point('telemetry')
                .field('altitude_ft', float(row['altitude_ft']))
                .field('airspeed_kts', float(row['airspeed_kts'])))
```

```

        .field('heading_deg', float(row['heading_deg']))
        .field('engine_rpm', float(row['engine_rpm']))
        .field('fuel_pct', float(row['fuel_pct']))
        .field('rate_of_climb', float(row['rate_of_climb']))
        .field('anomaly_score', float(anomaly_scores[idx]) if model else 0.0)
        .field('is_anomaly', bool(is_anomaly[idx]))
    write_api.write(INFLUX_BUCKET, INFLUX_ORG, point)
    if is_anomaly[idx]:
        with open('data/anomalies.log', 'a') as f:
            f.write(f"{row['timestamp_us']},{row['fault_flags']},{anomaly_scores[idx]:.4f}\n")
            print(f'Processed {len(df)} rows. Anomalies: {is_anomaly.sum()}')

def main():
    global last_row
    print('Pipeline running. Polling CSV every 6 seconds...')
    while True:
        try:
            df = pd.read_csv(CSV_PATH)
            new = df.iloc[last_row:]
            if len(new) > 0:
                process_batch(new.copy())
                last_row = len(df)
        except FileNotFoundError:
            print('Waiting for telemetry.csv...')
            time.sleep(6)

if __name__ == '__main__':
    main()

```

Learning moment

The pipeline uses a polling pattern — it checks the CSV for new rows every 6 seconds. In a production system you would replace this with a message queue like Kafka or ZeroMQ. But the polling pattern is simpler to set up and clearly illustrates the separation of concerns: the C++ layer owns writing, the Python layer owns reading. This is the same producer-consumer pattern used in flight data recorder ground station software.

Days 3–5

Train Isolation Forest anomaly detection model

~4 hrs

Step 14: Generate training data

You need clean, normal-flight data to train on. Run the simulator without any fault flags for 10 minutes to generate your training dataset:

```
# Terminal 1 - Start the C++ receiver first
cd build && ./aerosentinel

# Terminal 2 - Run simulator for 10 minutes (normal flight only)
python3 scripts/simulator.py
# Let it run for at least 600 seconds (~6000 packets)
# Press Ctrl+C when done

# You should now have data/telemetry.csv with ~6000 rows
# Check: wc -l data/telemetry.csv
```

Step 15: Train the model

Create `scripts/train_model.py`. This script trains the anomaly detector on your clean data and saves the model:

```
import pandas as pd
import numpy as np
import joblib
import os

from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

df = pd.read_csv('data/telemetry.csv')
print(f'Training on {len(df)} rows of normal flight data')

# Compute derived features
df = df.sort_values('timestamp_us')
df['dt'] = df['timestamp_us'].diff() / 1e6
df['rate_of_climb'] = df['altitude_ft'].diff() / df['dt'].replace(0, np.nan)
df['heading_change_rate'] = df['heading_deg'].diff().abs() / df['dt'].replace(0, np.nan)
df = df.fillna(0)

FEATURES = ['altitude_ft', 'airspeed_kts', 'engine_rpm',
            'rate_of_climb', 'heading_change_rate']
X = df[FEATURES].values

# Pipeline: scale features first, then train Isolation Forest
# contamination=0.02 means we expect ~2% of data to be anomalous
model = Pipeline([
    ('scaler', StandardScaler()),
    ('clf', IsolationForest(n_estimators=200, contamination=0.02,
                           random_state=42))
])
```

```
model.fit(X)

os.makedirs('models', exist_ok=True)
joblib.dump(model, 'models/anomaly_model.joblib')
print('Model saved to models/anomaly_model.joblib')

# Quick sanity check on the training data itself
preds = model.predict(X)
anomaly_rate = (preds == -1).mean()
print(f'Anomaly rate on training data: {anomaly_rate:.1%} (expected ~2%)')
```

```
# Run the training script
python3 scripts/train_model.py

# Expected output:
# Training on 6000 rows of normal flight data
# Model saved to models/anomaly_model.joblib
# Anomaly rate on training data: 2.0% (expected ~2%)
```

Step 16: Test anomaly detection against all four fault types

Restart the pipeline, then run the simulator with each fault flag and verify detection:

```
# Terminal 1: Start C++ receiver
./aerosentinel

# Terminal 2: Start Python pipeline (model now loads automatically)
python3 scripts/pipeline.py

# Terminal 3: Test each fault type
python3 scripts/simulator.py --fault engine_spike
# Check pipeline output - should show: Anomalies: N

python3 scripts/simulator.py --fault altitude_drop
python3 scripts/simulator.py --fault gps_freeze
python3 scripts/simulator.py --fault sensor_dropout

# Check anomaly log
cat data/anomalies.log
```

Learning moment

Isolation Forest works by building random decision trees and measuring how quickly each data point gets 'isolated' into its own leaf. Normal points sit in dense regions and take many splits to isolate. Anomalies sit in sparse regions and are isolated quickly. It is an

unsupervised method — you train it only on normal data and it learns what 'normal' looks like. This is exactly how aircraft health and usage monitoring systems (HUMS) work: you baseline normal behavior and flag deviations.

Days 5–7**FastAPI REST backend**

~3 hrs

Step 17: Write the FastAPI backend

Create scripts/api.py. This gives the dashboard and any external tools a clean interface to query the pipeline:

```
import os, sqlite3
from fastapi import FastAPI, HTTPException, Security
from fastapi.security import APIKeyHeader
from influxdb_client import InfluxDBClient

app = FastAPI(title='AeroSentinel API', version='1.0')
API_KEY = os.environ.get('AERO_API_KEY', 'aerosentinel-dev-key')
api_key_hdr = APIKeyHeader(name='X-API-Key', auto_error=False)

INFLUX_URL = 'http://localhost:8086'
INFLUX_TOKEN = os.environ.get('INFLUX_TOKEN', 'YOUR_TOKEN_HERE')
INFLUX_ORG = 'aerosentinel-org'

def require_key(key: str = Security(api_key_hdr)):
    if key != API_KEY:
        raise HTTPException(status_code=403, detail='Invalid API key')

@app.get('/status')
def status():
    return {'status': 'online', 'service': 'AeroSentinel', 'version': '1.0'}

@app.get('/anomalies')
def get_anomalies(_ = Security(require_key)):
    client = InfluxDBClient(url=INFLUX_URL, token=INFLUX_TOKEN, org=INFLUX_ORG)
    query = '''from(bucket:"aerosentinel")
    |> range(start: -1h)
    |> filter(fn: (r) => r._field == "is_anomaly" and r._value == true)
    |> limit(n: 50)'''
    tables = client.query_api().query(query)
    result = [{'time': r.get_time().isoformat(), 'value': r.get_value()}
              for t in tables for r in t.records]
    return {'count': len(result), 'anomalies': result}
```

```
@app.get('/telemetry/latest')
def get_latest(_ = Security(require_key)):
    client = InfluxDBClient(url=INFLUX_URL, token=INFLUX_TOKEN, org=INFLUX_ORG)
    query = '''from(bucket:"aerosentinel")
    |> range(start: -30s)
    |> filter(fn: (r) => r._field == "altitude_ft" or r._field == "airspeed_kts")
    |> last()'''
    tables = client.query_api().query(query)
    result = {r.get_field(): r.get_value() for t in tables for r in t.records}
    return {'latest': result}

# Run with: uvicorn scripts.api:app --reload --port 8000
```

```
# Start the API server
uvicorn scripts.api:app --reload --port 8000

# Test it - open in browser: http://localhost:8000/docs
# The /docs page is auto-generated by FastAPI - screenshot this for your portfolio

# Test with curl:
curl http://localhost:8000/status
curl -H 'X-API-Key: aerosentinel-dev-key' http://localhost:8000/telemetry/latest
```

Learning moment

FastAPI automatically generates an OpenAPI specification and a Swagger UI at /docs. This means your API is self-documenting — any developer can visit /docs and immediately understand every endpoint, its parameters, and its response schema. This is the standard in production APIs. Take a screenshot of the /docs page and include it in your README — it demonstrates professional API design discipline.

Deliverable: Live anomaly detection running. All four fault types detected. InfluxDB stores all frames. FastAPI running with Swagger docs at localhost:8000/docs. Anomalies logged to anomalies.log. Committed to GitHub.

Section 5: Week 3 — Dashboard, Docker & Portfolio Polish

The final week transforms a working project into a polished portfolio piece. This week determines whether a hiring engineer spends 30 seconds or 5 minutes on your GitHub. Everything here is designed to maximize that time.

Week 3 Goal

By the end of this week: Grafana dashboard auto-refreshing with live telemetry and anomaly markers. docker compose up launches the entire system from a cold clone. GitHub Actions CI runs on every push. README has an architecture diagram and screenshots. Repository tagged v1.0.

Days 1–3

Grafana live dashboard

~4 hrs

Step 18: Start Grafana with Docker

```
docker run -d \
  --name grafana \
  -p 3000:3000 \
  -e GF_SECURITY_ADMIN_PASSWORD=aerosentinel \
  grafana/grafana:10.2.0

# Open http://localhost:3000
# Login: admin / aerosentinel
```

Step 19: Connect Grafana to InfluxDB

In the Grafana web UI, navigate to Connections > Data sources > Add data source:

Setting	Value
Type	InfluxDB
Query Language	Flux
URL	http://localhost:8086
Organization	aerosentinel-org
Token	(paste the token you generated in Step 12)
Default Bucket	aerosentinel

Click Save & Test. You should see a green checkmark confirming the connection.

Step 20: Build the dashboard

Create a new dashboard with these five panels. For each, use the Flux query shown:

Panel 1: Altitude Time Series

```
from(bucket: "aerosentinel")
  |> range(start: -5m)
  |> filter(fn: (r) => r._field == "altitude_ft")
  |> aggregateWindow(every: 1s, fn: mean, createEmpty: false)
```

Visualization type: Time series. Title: Altitude (ft). Set auto-refresh to 2s.

Panel 2: Airspeed Gauge

```
from(bucket: "aerosentinel")
  |> range(start: -10s)
  |> filter(fn: (r) => r._field == "airspeed_kts")
  |> last()
```

Visualization type: Gauge. Min: 0, Max: 400. Add thresholds: green < 300, orange < 350, red >= 350.

Panel 3: Engine RPM

```
from(bucket: "aerosentinel")
  |> range(start: -5m)
  |> filter(fn: (r) => r._field == "engine_rpm")
```

Visualization type: Time series. Add a threshold line at 4000 RPM in red.

Panel 4: Anomaly Events (most important panel)

```
from(bucket: "aerosentinel")
  |> range(start: -5m)
  |> filter(fn: (r) => r._field == "is_anomaly")
  |> map(fn: (r) => ({r with _value: if r._value then 1.0 else 0.0}))
```

Visualization type: State timeline or Bar chart. Use red (#C0392B) for anomaly state. This is the panel to screenshot — trigger an anomaly with the simulator and capture the red markers appearing.

Panel 5: Anomaly Counter Stat

```
from(bucket: "aerosentinel")
  |> range(start: -1h)
  |> filter(fn: (r) => r._field == "is_anomaly" and r._value == true)
  |> count()
```

Visualization type: Stat. Label: 'Anomalies (last hour)'. Color: red when > 0.

Learning moment

Grafana is used for operational monitoring at Boeing, SpaceX, NASA's Jet Propulsion Laboratory, and virtually every aerospace company running modern ground station software. Being able to build and provision a Grafana dashboard — especially one connected to a time-series database — is a concrete, demonstrable skill. Many candidates who apply to Boeing list 'monitoring' on their resume but cannot actually build a dashboard from scratch.

Step 21: Export dashboard for auto-provisioning

In Grafana, go to your dashboard > Share (top right) > Export > Save to file. Save it as `docker/grafana/dashboards/aerosentinel.json`. This means anyone who clones your repo and runs Docker Compose gets the dashboard automatically.

Days 3–5**Docker Compose full system deployment**

~3 hrs

Step 22: Write `docker-compose.yml`

Create `docker/docker-compose.yml`. This is the file that makes your entire system launchable with one command:

```
version: '3.8'

services:
  influxdb:
    image: influxdb:2.7
    ports: ['8086:8086']
    volumes:
      - influxdb-data:/var/lib/influxdb2
      - ./influxdb/config:/etc/influxdb2
    environment:
      - DOCKER_INFLUXDB_INIT_MODE=setup
      - DOCKER_INFLUXDB_INIT_USERNAME=admin
      - DOCKER_INFLUXDB_INIT_PASSWORD=aerosentinel123
      - DOCKER_INFLUXDB_INIT_ORG=aerosentinel-org
      - DOCKER_INFLUXDB_INIT_BUCKET=aerosentinel
      - DOCKER_INFLUXDB_INIT_ADMIN_TOKEN=aerosentinel-init-token

  grafana:
    image: grafana/grafana:10.2.0
    ports: ['3000:3000']
    volumes:
      - grafana-data:/var/lib/grafana
```

```
- ./grafana/provisioning:/etc/grafana/provisioning
- ./grafana/dashboards:/var/lib/grafana/dashboards
environment:
  - GF_SECURITY_ADMIN_PASSWORD=aerosentinel
depends_on: [influxdb]

pipeline:
  build:
    context: ..
    dockerfile: docker/Dockerfile.pipeline
  volumes: ['../data:/app/data', '../models:/app/models']
  environment:
    - INFLUX_TOKEN=aerosentinel-init-token
  depends_on: [influxdb]

api:
  build:
    context: ..
    dockerfile: docker/Dockerfile.pipeline
  command: uvicorn scripts.api:app --host 0.0.0.0 --port 8000
  ports: ['8000:8000']
  environment:
    - INFLUX_TOKEN=aerosentinel-init-token
    - AERO_API_KEY=aerosentinel-dev-key
  depends_on: [influxdb]

volumes:
  influxdb-data:
  grafana-data:
```

Step 23: Write the Python service Dockerfile

Create docker/Dockerfile.pipeline:

```
FROM python:3.11-slim

WORKDIR /app

# Copy and install dependencies first (cached if requirements.txt unchanged)
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy application code
COPY scripts/ ./scripts/
COPY models/ ./models/
```

```
CMD ["python3", "scripts/pipeline.py"]
```

```
# Test cold-start from your docker/ directory:
docker compose down -v          # Wipe everything
docker compose up --build       # Rebuild and start fresh

# All services should start without any manual steps.
# Grafana should show your dashboard at http://localhost:3000
```

Days 5–7

GitHub Actions CI + README + v1.0 release

~3 hrs

Step 24: Write the GitHub Actions CI workflow

Create `.github/workflows/ci.yml`. This runs your tests automatically every time you push to main:

```
name: CI

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  cpp-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Install dependencies
        run: sudo apt-get install -y cmake g++ valgrind
      - name: Configure
        run: cmake -B build -DCMAKE_BUILD_TYPE=Debug
      - name: Build
        run: cmake --build build -j4
      - name: Run tests
        run: cd build && ctest --verbose

  python-checks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }
```

```
- name: Install packages
  run: pip install -r requirements.txt
- name: Import check
  run: python3 -c "import pandas, sklearn, influxdb_client, fastapi;
print('OK')"
```

Step 25: Write the SECURITY.md

Create SECURITY.md at the project root. This demonstrates security awareness — a notable differentiator for defense roles:

```
# Security Notes

## Current implementation (development)
- API key authentication on all non-public endpoints
- Input validation on all FastAPI routes (Pydantic)
- Pinned Docker image versions (supply chain)
- Pinned Python package versions in requirements.txt
- CRC-16 checksum validation on all telemetry packets

## Production hardening (not implemented, would be required for deployment)
- Replace static API key with JWT tokens (RS256 signed)
- Add TLS/mTLS on all service-to-service communication
- Store secrets in Vault or AWS Secrets Manager, not environment variables
- Add rate limiting on all API endpoints
- Implement RBAC (role-based access control) for multi-user environments
- Container hardening: non-root user, read-only filesystem, no-new-privileges
- In a classified environment: replace InfluxDB OSS with a FedRAMP-authorized service
- Compliance note: for airborne software, CRC-16 would be replaced with DO-178C compliant error detection per DAL (Design Assurance Level) requirements
```

Step 26: Write the README and tag v1.0

Your README is the first thing a hiring engineer sees. Structure it as follows:

- Project title + one-sentence description
- Live demo screenshot (your Grafana dashboard with anomalies visible)
- Architecture diagram (create in draw.io or Excalidraw, export as PNG)
- Tech stack badges (use shields.io — a C++17 badge, Python badge, Docker badge)
- Quick start section: exactly three commands to get the system running
- How it works: one paragraph per layer with links to key source files
- Testing: how to run the test suite and inject faults

```
# Tag the release when README is complete
```

```
git tag -a v1.0 -m "AeroSentinel v1.0 - Initial release"
git push origin v1.0
```

```
# Create a GitHub Release from the tag:
# Go to your repo > Releases > Create release from tag
# Write release notes summarizing what was built
```

Deliverable: docker compose up launches the full system from a cold clone. Grafana dashboard live with anomaly markers. GitHub Actions CI shows green badge. README with architecture diagram, screenshot, and 3-command quickstart. Repo tagged v1.0 with a GitHub Release.

Section 6: Resume, Interview Prep & Bonus Features

6.1 Resume bullet points

Use these as a template — customize the numbers as you build. The goal is specificity; vague bullets get skipped.

C++ Systems Engineering
Developed a real-time avionics telemetry pipeline in C++17, receiving binary UDP packets at 10Hz with CRC-16 checksum validation, a fixed-size ring buffer, and CSV persistence — validated with Google Test and zero memory errors under AddressSanitizer
Machine Learning / Data Engineering
Built a Python anomaly detection engine using Isolation Forest (scikit-learn) trained on 5 derived telemetry features, achieving detection of 4 injected flight fault types (engine spike, altitude drop, GPS freeze, sensor dropout) with <2% false positive rate on normal data
DevOps / Infrastructure
Containerized a multi-service monitoring stack (InfluxDB, Grafana, FastAPI) with Docker Compose and auto-provisioned Grafana dashboards, with a GitHub Actions CI pipeline running C++ and Python tests on every commit to main
Security
Implemented API key authentication on all protected endpoints and authored a SECURITY.md outlining production hardening steps (JWT, mTLS, RBAC, FedRAMP compliance) aligned with NIST SP 800-53 controls for defense-adjacent systems

6.2 Interview preparation

Boeing defense software interviews will test your depth on three topics from this project. Prepare answers to these questions:

On the C++ ring buffer

Question	Key points for your answer
Why a ring buffer instead of a queue?	O(1) push/pop, no dynamic allocation, predictable memory usage, never blocks — critical in real-time systems
What happens when it fills up?	Oldest entry is overwritten — for a telemetry system this is correct because recent data is more valuable than old data

What would you change for thread safety?	Add a mutex around push/pop, or use std::atomic for the head/tail indices (lock-free approach)
--	--

On the Isolation Forest model

Question	Key points for your answer
Why unsupervised instead of supervised?	You rarely have labeled 'anomalous' flight data. The model learns normal patterns and flags deviations — same as real HUMS systems
What is contamination?	The expected fraction of anomalies in training data. Set too low: misses anomalies. Too high: too many false positives
How would you improve it in production?	Online learning to adapt to aircraft aging, ensemble with rule-based checks (RPM > 4500 always bad), add more features

On deployment and security

Question	Key points for your answer
Why Docker Compose?	Reproducible builds — anyone can run the full system with one command. Eliminates 'works on my machine' issues
What would you change for production?	Kubernetes for orchestration, Vault for secrets, mTLS between services, read-only containers
What is DO-178C?	Avionics software certification standard. Define DALs, require requirements traceability, formal verification. CRC-16 would need to meet DAL-specific error coverage requirements

6.3 Bonus features to add if you finish early

These are listed in order of resume impact. Each one adds a concrete talking point without blowing the 3-week budget.

Bonus 1: Protocol Buffers serialization (high impact, ~3 hrs)

Replace the CSV pipeline with protobuf serialization. Define a .proto schema, use protoc to generate C++ and Python code, and have the C++ layer write binary protobuf frames that the Python layer deserializes. This is the format used in real avionics ground station pipelines and is a significant resume differentiator.

```
# Install protobuf compiler
brew install protobuf # Mac
sudo apt install protobuf-compiler # Linux
pip install protobuf
```

```
# Define your schema in telemetry.proto
# Run: protoc --python_out=scripts/ --cpp_out=src/ telemetry.proto
```

Bonus 2: PyQt5 desktop GUI (portfolio wow factor, ~4 hrs)

Build a simple desktop application with PyQt5 that shows a live altitude/airspeed plot and a red ANOMALY DETECTED banner when an anomaly fires. This adds a visual demo component that photographs well and impresses non-technical interviewers.

```
pip install pyqt5 pyqtgraph

# Key components:
# - QMainWindow as the base
# - pyqtgraph PlotWidget for the live chart
# - QTimer polling InfluxDB every 2 seconds
# - QLabel with red background for anomaly alert
```

Bonus 3: Kalman filter for GPS smoothing (technical depth, ~2 hrs)

Add a 1D Kalman filter in Python that smooths the GPS altitude signal. This algorithm is used in every real avionics system and comes up in nearly every robotics and aerospace interview. A candidate who can explain the predict-update cycle of a Kalman filter from their own project code is rare.

```
class KalmanFilter1D:
    def __init__(self, process_noise=1.0, measurement_noise=10.0):
        self.x = 0.0 # State estimate
        self.P = 1.0 # Estimate uncertainty
        self.Q = process_noise # Process noise covariance
        self.R = measurement_noise # Measurement noise covariance

    def update(self, measurement: float) -> float:
        # Predict
        P_pred = self.P + self.Q
        # Update
        K = P_pred / (P_pred + self.R) # Kalman gain
        self.x = self.x + K * (measurement - self.x)
        self.P = (1 - K) * P_pred
        return self.x
```

Bonus 4: Demo video (highest ROI, ~1 hr)

Record a 2-minute Loom video showing the full system running: simulator sending packets, C++ receiver printing validated frames, the pipeline detecting an injected fault, and the Grafana dashboard showing the anomaly markers appear in real time. Link this in your README and LinkedIn profile. Most candidates do not do this. It makes your project unforgettable.

6.4 Final checklist before applying

- ✓ GitHub repository is public with a clean commit history (not one giant initial commit)
- ✓ README has an architecture diagram, dashboard screenshot, and 3-command quickstart
- ✓ docker compose up runs cleanly from a fresh clone with no manual steps
- ✓ GitHub Actions CI badge is green
- ✓ Repository tagged v1.0 with a GitHub Release and release notes
- ✓ You can explain every file and every design decision without referencing notes
- ✓ You have answers ready for: ring buffer design, Isolation Forest, CRC validation, Docker, and DO-178C
- ✓ SECURITY.md is present and references real defense security concepts
- ✓ requirements.txt has pinned exact versions
- ✓ Resume bullets are updated with specific numbers from your build

You are now ready.

Build it. Understand it. Own it.